marwanelgendy.link

Site Language, Detail-Modal Case Studies & AI Agent Knowledge Base

Completed copy, content specification, and open punch list

Prepared for Marwan Elgendy · Portfolio + AI chat agent
Repository: github.com/geda0/gvp (branch: main)
May 2026

Contents

1. Purpose & how to use this document

This document gathers everything identified in the most recent review of the live site and AI chat agent into a single actionable package. It does three things: it completes the site's written language so every section is finished in one consistent voice; it provides full case-study content for the portfolio and playground detail modals, which are currently empty stubs; and it specifies the enhancements the chat agent's knowledge base needs so its answers match the substance in Marwan's resume and Q&A material.

The document is organized so each piece is drop-in ready. Sections 3 through 7 contain final copy in tinted boxes labeled FINAL COPY — that text is meant to be used as written. Section 8 specifies the consolidation of the hero mascot and chat agent into one morphing element. Section 9 specifies the agent knowledge base as a content spec to verify against the actual data files in the repository. Section 10 is the open punch list: visual and technical issues that copy alone cannot fix, in priority order.

A note on sourcing

The completed copy and case studies draw on three sources that were available for this review: the public resume PDF, the question-and-answer material Marwan provided, and the live site behavior captured in screenshots. The agent knowledge-base section (8) is written as a specification to validate against the repository's data files rather than as a direct edit, because the knowledge files under the repo's data directory were not directly readable during this review. Every claim in the agent spec traces back to the resume or the Q&A material — nothing is invented.

2. The voice — defined once, applied everywhere

Everything in this document follows the voice the existing hero and pillar cards already established. Defining it explicitly keeps the completed sections indistinguishable from what shipped.

Voice characteristics

- Declarative and concrete. Short sentences. Statements, not adjectives doing a metric's job.
- Specific about scale. "Thousands to millions of subscribers" — not "massive scale." Numbers carry the weight.
- Outcome-led. Each unit of copy ends on a result, ideally quantified.
- Quietly confident. The work speaks; the copy doesn't oversell. "Systems that hold up in production," not "robust, world-class systems."
- Third person for the agent, first person for the site. The site is Marwan speaking ("I build systems..."). The agent speaks about Marwan ("Marwan led a migration..."). This boundary is never crossed.

Quick test

If a sentence could appear on any senior engineer's portfolio, it is not finished. If it names a system, a constraint, and a result, it is. Apply this test to every line below before shipping it.

3. Site language — what exists, what was missing

The audit below maps every section of the site against its completion state at the time of review.

State	Section	Notes
Complete	Hero — eyebrow, name, title, tagline, proof line, company strip	On-voice. No copy changes needed; see Section 10 for layout issues.
Complete	Three pillar cards	Platform Architecture / End-to-End Delivery / Data-Intensive SaaS. On-voice.
Partial	Portfolio cards — Apptio, JumpCloud, HP	Four-part structure written and live. AT&T card drafted; confirm it is deployed.
Missing	Earlier-career line	Roles before 2017 (Sunrise Resorts, OIG, 5D Agency) have no presence. Timeline looks like it starts mid-career.
Partial	Playground cards	Titles and one-line descriptions exist; no consistent structure. Does not match portfolio card rhythm.
Missing	Portfolio detail modals	Empty stub: "Tech & tools" + "Open link". Section 5 fills these.
Missing	Playground detail modals	Same empty stub. Section 6 fills these.
Missing	About / experience summary block	No on-page career summary. Copy exists unused in Q&A material. Section 4 supplies it.
Partial	Contact form	Fields exist; no framing copy, no confirmation-state message. Section 7 supplies both.
Missing	Chat panel empty-state	Panel opens blank with no greeting. Section 7 supplies the empty-state line.
Partial	Spaceman thought-bubble phrases	Ambient and chat-state phrases exist; streaming state missing; could use variety. Section 7.
Spec	Agent knowledge base	Answers well but can be sharper. Full spec in Section 9.

4. New section — About / experience summary

The site currently routes all career depth to a PDF or the chat agent. A short, scannable on-page summary means a recruiter skimming on a phone gets the whole picture without a download and without engaging the bot. This block belongs between the hero and the Portfolio section.

The copy below is lightly edited from Marwan's own Q&A material so it sits in the same voice as the rest of the site.

FINAL COPY — ABOUT BLOCK

15+ years building scalable systems and data platforms.

My work spans cloud-native architecture on AWS, Kubernetes-based platforms, large-scale data processing on Spark, EMR, and Databricks, and distributed microservices. I've led complex initiatives end to end — from system design and infrastructure automation through to mentoring engineers and raising the technical bar across teams.

I work with AI-assisted tooling daily: planning large initiatives and generating documentation with Claude, prototyping and writing code in Cursor, and using multi-model workflows to move faster without lowering the bar.

Why the AI-tooling line stays

For the roles Marwan is targeting, "uses AI tooling deliberately and well" is a current, relevant signal — and it is true to how he works. It also quietly explains the existence of the chat agent itself: the site is built by someone who builds with these tools.

5. Portfolio detail modals — full case studies

The portfolio cards say "View details · link inside," but the modal behind them is an empty stub. A click that leads nowhere is worse than no click. This section fills each modal with a real case study.

Each case study follows the same five-part structure so the modals share a rhythm: Context (the situation), The problem (what made it hard), What Marwan did (the technical substance), The outcome (the result), and Reflection (a short, senior note on a tradeoff or what he'd revisit). The Reflection section is what separates a case study from a bullet list — it shows judgment, not just activity.

5.1 Apptio (an IBM company) — detail modal

FINAL COPY — APPTIO (AN IBM COMPANY) DETAIL MODAL

Apptio (an IBM company)

Senior Software Engineer · 2023–2025

Migrating financial data pipelines to a higher-performance architecture — with zero tolerance for correctness drift.

Context

Apptio's data-processing pipelines handled high-volume usage and financial datasets — the kind of data where a customer-facing number being slightly wrong is not a bug, it is a broken product. The pipelines ran on Kubernetes and Java. As volume grew, they were hitting performance limits: some runs stretched across days, and the cost of running them was climbing in step with the data.

The problem

The obvious fix — moving the heavy workloads to a faster processing engine — carried a real risk. Financial datasets have zero tolerance for drift. A migration that was 10x faster but introduced even small discrepancies would be worthless, and a big-bang cutover would make any discrepancy hard to isolate. The challenge was not raw performance. It was earning enough confidence in the new system to trust it with money.

What Marwan did

He migrated the targeted heavy workloads from Kubernetes and Java to EMR Spark with Scala, chosen for throughput on large datasets. Crucially, he built a parallel reconciliation pipeline: the legacy and new systems ran side by side on the same inputs, and the pipeline flagged any discrepancy between their outputs in real time. That reconciliation harness turned a risky migration into an incremental one — workloads could be cut over a piece at a time, each piece verified against the system it replaced before the old path was retired. Alongside the migration he built automated operational workflows using Python, AWS Lambda, and Step Functions to reduce manual intervention, and used Terraform to provision and manage the AWS environment as code.

The outcome

The migration delivered roughly 10x faster processing while maintaining service availability throughout. Most importantly, it completed with zero correctness drift on the financial datasets — the reconciliation pipeline provided the evidence that made an incremental, low-risk cutover possible.

Reflection

The reconciliation pipeline was extra work that produced no customer-facing feature — and it was the most important part of the project. It is the clearest example of a principle Marwan applies deliberately: on correctness-critical systems, the migration is the easy half; the proof that the migration is correct is the half that actually matters. Given the same problem again, he would build the reconciliation harness first, before touching the workloads.

Tech: Java · Scala · EMR Spark · Kubernetes · AWS Lambda · Step Functions · Terraform · Databricks · Snowflake

5.2 Hewlett Packard (HP) — detail modal

FINAL COPY — HEWLETT PACKARD (HP) DETAIL MODAL

Hewlett Packard (HP)

Specialist Software Engineer · 2018–2021

Scaling HP's Instant Ink subscription service from thousands of customers to millions — in months, not years.

Context

HP Instant Ink is HP's printer-ink subscription service. The business needed it to grow from serving thousands of customers to serving millions, globally. The system underneath it was a Ruby on Rails monolith — fine at its original scale, but not a foundation that could absorb that kind of growth without buckling.

The problem

A monolith does not scale gracefully by orders of magnitude. The work was not a tuning exercise; it was a structural transformation — and it had to happen on a live, revenue-generating service without interrupting customers, and on a timeline measured in months. It also had to happen across four geographically distributed teams, which made clear boundaries and shared architectural direction a hard requirement, not a nice-to-have.

What Marwan did

He planned and led the transformation, applying Domain-Driven Design to decompose the Rails monolith into cloud-native microservices and data pipelines on AWS — Lambda, API Gateway, SNS, SQS, DynamoDB, Redshift, CloudFormation. Domain-Driven Design did double duty here: it produced the service boundaries the architecture needed, and it gave four distributed teams a shared language for who owned what. He drove Test-Driven Development and agile practices across those teams, working closely with DevOps, QA, and Product Management to keep the migration stable as it progressed.

The outcome

Instant Ink scaled from thousands to millions of subscribers worldwide. The transformation was completed in a matter of months, and service reliability and stability were maintained throughout — customers experienced growth, not disruption.

Reflection

The technically interesting part is the microservices migration. The genuinely hard part was coordination: four distributed teams, one live service, a tight clock. Domain-Driven Design is often discussed as a modeling technique — here it was just as much an organizational tool. The boundaries it drew were the same boundaries the teams worked along, which is why the migration could move quickly without the teams colliding.

Tech: Ruby on Rails · ReactJS · AWS Lambda · API Gateway · SNS · SQS · DynamoDB · Redshift · CloudFormation · Docker · Jenkins · Chef · Splunk

5.3 JumpCloud — detail modal

FINAL COPY — JUMPCLOUD DETAIL MODAL

JumpCloud

Senior Software Engineer · 2022–2023

A Microsoft M365 integration for an identity platform serving 50,000+ tenants — built on a data layer that had to satisfy two contradictory requirements at once.

Context

JumpCloud is a cloud directory platform. Marwan directed the end-to-end technical design of a Microsoft M365 integration that expanded the platform's capabilities and enabled cross-platform compatibility. The integration was exposed through a GraphQL API and had to serve a multi-tenant production environment of more than 50,000 tenants.

The problem

The data layer had to do two things that pull in opposite directions. It had to preserve fast, relational queries over the core identity graph — the structured, related data at the heart of a directory product. And it had to absorb a high volume of writes for per-tenant custom attributes and hierarchies, which are flexible, variable, and tenant-specific by nature. A purely relational store would strain under the write volume and the schema variability; a purely document store would give up the relational query performance the identity graph depends on.

What Marwan did

He architected a hybrid data layer that used each database for what it is good at. The core identity graph lived in PostgreSQL, keeping relational queries fast and consistent. Per-tenant custom attributes and hierarchies were isolated in MongoDB, where their variability and write volume were a natural fit rather than a fight. On top of this he designed the multi-tenant GraphQL schema and resolvers for the production environment, and delivered the integration end to end using Docker, Golang, gRPC, JavaScript, and Node.js.

The outcome

The integration shipped to a production environment of 50,000+ tenants. The hybrid data layer preserved relational query performance on the identity graph while absorbing high write volume across all tenants — the two requirements that were in tension were both met, because each was routed to the store suited to it.

Reflection

The instinct under deadline pressure is to pick one database and force the whole problem into it. The better answer here was to recognize that the problem genuinely had two shapes — a relational shape and a document shape — and to stop treating that as something to eliminate. The hybrid layer is more moving parts, but it is the design that stopped fighting the data. Knowing when added structure is worth it is the judgment call, and on a 50,000-tenant system it was.

Tech: Golang · gRPC · Node.js · JavaScript · PostgreSQL · MongoDB · GraphQL · Docker

6. Playground — card copy and detail modals

The Playground cards need a structure that parallels the portfolio cards so the site keeps one rhythm — but the frame has to be honest that these are personal experiments, not employed work. Portfolio cards use Role / Problem / Action / Outcome. Playground cards use What it is / What was explored / Takeaway. Same visual cadence; appropriate framing.

6.1 Playground card copy

FINAL COPY — MONDAY ROVER (RASPBERRY PI) CARD

Monday Rover (Raspberry Pi)

Personal project · robotics + computer vision

A Raspberry Pi rover built to explore real-time computer vision on constrained hardware.

Wired up the motor control, the camera pipeline, and an on-device vision loop — treating a hobby build with the same habits as production: version control, clear module boundaries, reproducible setup.

Takeaway: hands-on with embedded constraints, latency budgets, and the gap between “works on my desk” and “works reliably.”

Tech: Raspberry Pi · Python · OpenCV · Computer Vision

FINAL COPY — GENERATIVE VIDEO PLATFORM CARD

Generative Video Platform

Personal project · generative AI + product UX

An exploration of generative AI for video workflows — from the UX of prompting through to the platform that runs it.

Prototyped the end-to-end loop: the interface for shaping a generation, the orchestration behind it, and where the slow, expensive, or unpredictable parts need handling.

Takeaway: a working view of what it takes to put generative models behind a usable product, not just a demo.

Tech: Generative AI · Video · Platform · Product UX

Template for any additional Playground items

One line — what it is. One to two lines — what was explored. One line — Takeaway: what it demonstrates. Tech tags. Keep the spaceman's bubble phrase “Prototypes with production habits” in mind: that is the through-line of the whole section.

6.2 Playground detail modals

These modals carry lighter stakes than the portfolio case studies, but they should not be empty either. They use a shorter four-part structure: What it is / What I built / What I learned / Honest

limitations. The “Honest limitations” part matters — it is what makes an experiment read as the work of a senior engineer rather than a portfolio-padding demo.

FINAL COPY — MONDAY ROVER (RASPBERRY PI) DETAIL MODAL

Monday Rover (Raspberry Pi)

Personal project · robotics + computer vision

What it is

A small Raspberry Pi rover — motors, a camera, and an on-device vision loop — built as a deliberate exercise in running real-time computer vision on hardware that gives you very little room to be wasteful.

What I built

The full stack of a working rover: motor control, the camera capture pipeline, and a vision loop running on the Pi itself rather than offloaded to a server. The build was kept under version control with clear module boundaries and a reproducible setup, because the point was to practice production habits on a project small enough to see all of at once.

What I learned

Constrained hardware makes you honest about latency budgets and resource use in a way that cloud infrastructure rarely forces. The gap between a vision loop that works once on a desk and one that works reliably while the rover is moving is where the actual engineering lives.

Honest limitations

This is an exploration, not a product. It is single-purpose, the vision model is modest, and it has not been hardened for anything beyond a controlled environment. Its value is the hands-on intuition for embedded constraints, not the rover itself.

Tech: Raspberry Pi · Python · OpenCV · Computer Vision · Embedded

FINAL COPY — GENERATIVE VIDEO PLATFORM DETAIL MODAL

Generative Video Platform

Personal project · generative AI + product UX

What it is

An end-to-end exploration of what it takes to put generative video behind a usable product — not a single model call, but the whole loop from the prompting interface to the platform that runs the generation.

What I built

A prototype of the complete path: an interface for shaping a generation, the orchestration that runs it, and explicit handling for the parts of generative workflows that are slow, expensive, or unpredictable. The focus was the seams — where the model meets the product, and where a demo would quietly break if it had to serve a real user.

What I learned

The model is rarely the hard part. The product engineering around it is — managing latency and cost, setting expectations in the UI, and designing for outputs that vary run to run. Building the loop end to end surfaces those problems in a way that a standalone model demo never does.

Honest limitations

This is a prototype built to learn the shape of the problem, not a launched platform. It has not been load-tested or cost-optimized for production, and the UX is exploratory. What it produced is a grounded, first-hand view of the real work in productizing generative AI.

Tech: Generative AI · Video · Platform Orchestration · Product UX

7. Remaining site copy — earlier career, contact, chat, spaceman

7.1 Earlier-career line

Roles before 2017 currently have no presence on the site, which makes a 15-year career look like it began mid-stream. These roles should not become cards — that would dilute the senior-platform narrative — but they need a single quiet line below the portfolio cards so the timeline is whole.

FINAL COPY — EARLIER-CAREER LINE (PLAIN TEXT, BELOW PORTFOLIO CARDS)

Earlier: Software Development Manager at Sunrise Resorts, embedded Linux systems at OIG Electronics, and web and game development at 5D Agency — 2011–2016.

7.2 Contact form — framing and confirmation

The contact form has fields but no framing copy — nothing that tells a visitor what the form is for or what happens after they send it. Two additions: a subtitle under the heading, and a confirmation-state message. The confirmation copy is worth getting right: the form is backed by a durable AWS pipeline that persists the message before reporting success, so the confirmation can be quietly definite.

FINAL COPY — CONTACT FORM SUBTITLE (UNDER THE “CONTACT” HEADING)

Hiring, a role, or a project worth talking about — send a note and I'll get back to you. The fastest way to reach me.

FINAL COPY — CONTACT FORM CONFIRMATION STATE (AFTER SUCCESSFUL SUBMIT)

Got it — your message is on its way. I'll be in touch.

7.3 Chat panel — empty state

The chat panel currently opens blank. A cold open should show one assistant line plus the existing suggestion chips, so the panel feels intentional rather than unfinished. The line also reinforces the “grounded, not a general chatbot” framing at the moment the visitor is deciding whether to engage.

FINAL COPY — CHAT PANEL EMPTY-STATE LINE

Ask me about Marwan's work — the HP Instant Ink scale-up, the Apptio pipeline migration, the JumpCloud identity platform, or anything across his 15 years. Grounded in his resume, not a general chatbot.

7.4 Spaceman thought-bubble phrases

The spaceman's thought bubble is the site's status surface — it cycles ambient phrases when idle and switches to chat-state phrases when the agent is working. The ambient set is in good shape. Two gaps: there is a “Thinking…” phrase for the pre-token wait but none for while the agent is

actively streaming a reply, and the ambient set could use a little more variety. Additions below, all in the established style.

FINAL COPY — SPACEMAN PHRASE ADDITIONS

Streaming state (currently missing):

- Pulling that together…

Additional ambient phrases (for variety):

- Correctness over speed.
- 15 years, still shipping.

Full chat-state mapping (for reference)

- idle — cycles ambient phrases
- sending — “Got your question…”
- thinking — “Thinking…” (exists)
- streaming — “Pulling that together…” (new — fills the gap)
- tool call — “Pulling that up…”
- error — “Hmm, let’s try that again.”

8. Hero & agent consolidation — one morphing element

Today the site has three separate surfaces doing one related job: the spaceman's thought bubble, the hero chat bar, and the chat modal. They are wired together — the bubble shows “Thinking…” while the agent works — but they remain visibly three things. A recent screenshot of the Portfolio page makes the cost concrete: the spaceman's thought bubble renders in one place while the slim chat bar is docked elsewhere, so both appear at once. The bubble is programmed globally; the bar is programmed globally; they were never merged.

This section specifies the consolidation: the bubble and the bar become a single element — call it the agentNode — and the modal becomes a true expansion of it rather than a separate panel. The result is one object at three sizes. The spaceman is always thinking; the bubble is that thought; the bar is the thought inviting you in; the modal is the thought, expanded. The hero stops announcing the agent and becomes the agent.

The core principle

One DOM element with three style states — not three elements that swap. The morph reads as continuous only if the bubble, the bar, and the modal's seed are the same node transitioning its own properties. The moment the implementation cross-fades between separate elements, the eye catches it and it is back to feeling like a swap.

8.1 The three states

State A — Bubble (idle)

The agentNode is the spaceman's thought bubble. It shows ambient phrases (“Building production systems,” “Correctness over speed”) and the agent-status phrases when relevant (“Thinking…”). Rounded bubble shape, with the dot-trail tethering it down to the spaceman. This replaces the separate floating bubble that currently sits beside the mascot.

State B — Bar (invited)

On hover or focus, the same element morphs: the speech-bubble shape eases into the pill-shaped input, the ambient text becomes the “Ask anything about my work” placeholder, the send arrow fades in. Same position, same width band. The bubble did not get replaced by a bar — it became one. The dot-trail can stay (now reading as “the spaceman is listening”).

State C — Modal (active)

Clicking or submitting eases the bar open into the existing chat modal — but the modal now visually grows out of the bar rather than appearing as a separate centered panel. The bar is the seed; the modal expands from it. Critically, the exact open/close behavior, timing, backdrop, Escape handling, and close button of the current modal are all preserved. Nothing about the modal's mechanics changes; only its origin point changes, so it feels born from the bubble instead of teleported in. Exiting eases the modal back down to the bar's footprint, the bar relaxes to the bubble, and the spaceman resumes ambient thoughts.

State priority — agent status always wins

When the chat is mid-request, chatBus events drive the displayed text regardless of state: the bubble shows “Thinking…,” and if the modal is open the in-panel typing indicator shows too. The morph state and the agent state must never fight.

8.2 Page-dependent resting state

The agentNode is one global element, present on every page — which matches what the thought bubble already does today. There is no “skip the bubble on sub-pages” exception. Instead, its resting state depends on the page, because the spaceman is present on every page but positioned differently.

Page	Resting state	Why
Home hero	State A — Bubble	The spaceman is large and central; the agentNode rests as its thought bubble, tethered by the dot-trail, and morphs to bar on hover or focus.
Sub-pages (Portfolio, Playground)	State B — Bar	The spaceman is present but smaller and off to the side. A sub-page visitor reading a project card is in “I might want to ask something” mode, not “admire the mascot” mode — so the agentNode rests as the bar. The dot-trail still connects it to the spaceman, so it is visibly still the mascot's thought, presented as an invitation.

This directly fixes the every-page duplication bug

The Portfolio screenshot showed the standalone bubble in one position and the slim chat bar in another, both visible. With one global agentNode there is only one thing to render — the duplication cannot occur. The resting state differs by page, but it is always the same single element tied to the spaceman.

While consolidating — fix the edge clipping

The same screenshot shows the bubble still clipping at the left viewport edge on narrower widths (“TBM SaaS —” starting hard against x=0). When the bubble and bar merge into the agentNode, give the merged element a max-width and edge-padding constraint so it never clips — regardless of which state it is in or where the spaceman sits.

8.3 Implementation plan

This is a consolidation of three surfaces into one morphing element — not a rebuild. The modal's internals, the streaming, the tool-call buttons, the footer actions, and the chatBus event system are all untouched.

1. Merge the bubble and the bar into one component

Today js/chat.js has a launcher (the hero bar) and the spaceman has a separate thought-bubble element. Make them one — the agentNode — living in the chat bar's current position. It carries a data-state attribute: bubble, bar, or modal. All three looks are CSS driven off that attribute on the same element.

2. Bubble-to-bar morph is pure CSS transition

The properties that animate between State A and State B: border-radius (16px bubble corners to 999px pill), background and border (soft bubble fill to translucent input with the accent ring), width (bubble hugs its text to bar fixed-width band), and padding. The text content swaps at the transition midpoint (ambient phrase to placeholder); the send arrow fades in. Because it is one element transitioning its own box, the browser tweens it for free. Trigger: mouseenter / focus sets data-state to bar; mouseleave without focus returns it to bubble.

3. Bar-to-modal expansion — the part that needs care

Keep the existing modal element and all its current logic — backdrop, Escape handler, close button, focus trap, open/close timing. Change only its transform-origin. Instead of the modal animating in from screen-center, it animates from the agentNode's bounding rect: on open, capture the bar's getBoundingClientRect(), set the modal's transform-origin to that point, and have it scale and expand from there to its resting size and position. On close, reverse — it collapses back toward the bar's footprint, then the bar relaxes to bubble. The modal's mechanics are untouched; only where its open/close animation originates changes.

4. The dot-trail stays mounted across all three states

It is the tether that keeps “this is the mascot's thought” true even when the thought is a full modal. Fade it slightly in modal state (it is visually busy enough); keep it crisp in bubble and bar states.

What changes, file by file

- **js/chat.js —** merge launcher and thought-bubble into one agentNode component; add the data-state machine; rewrite the modal's open/close to capture and animate from the bar's rect instead of screen-center.
- **css/chat.css —** the three-state styling driven off [data-state] on the one element; the transform-origin-based modal expansion; transition timings matched to the modal's current open/close duration so nothing feels re-tuned.
- **spaceman code —** remove the standalone thought-bubble element — its job moves to the agentNode. Keep the dot-trail; re-anchor it to point at the agentNode's position.

Relationship to the hero-awareness work

This consolidation and the chatBus hero-awareness wiring are complementary, not competing. chatBus is the event system that tells the agentNode what to display (Thinking, Streaming, error). This section is about what the agentNode physically is — one morphing element instead of three. Build chatBus first if it is not already in place; the agentNode subscribes to it exactly as the standalone bubble was specified to.

9. AI agent knowledge base — enhancements & edits

The agent answers well today — its replies are in third person, accurate, and on-voice (the live EMR Spark answer is a good example). The enhancements below are about making it sharper and more reliably consistent, and about hardening the few sensitive questions so the agent never improvises on them. This section is written as a content specification to validate against the repository's knowledge files; the structure references the bio / roles / faq model the knowledge pack was designed around.

How to apply this section

For each item: locate the corresponding record in the knowledge files under the repo's data directory, and confirm it carries the detail and framing specified here. Where a record does not exist, add it. Every fact below traces to the resume or Marwan's Q&A material — the spec adds no new claims, it only ensures the agent has the strongest accurate version of each.

9.1 Sharpen the two flagship stories

The HP scale-up and the Apptio migration are the agent's most-asked, highest-value answers. Marwan's Q&A material describes both in richer detail than the resume — and that extra detail is what makes the answers concrete. The roles records for HP and Apptio should carry these specifics in their highlights.

Apptio role record — highlights should include

- Framing: it was a scaling AND cost problem — pipelines were “running for days and becoming expensive to run.” Not just “migrated workloads.”
- The reconciliation pipeline ran the legacy and new systems in parallel and flagged discrepancies in real time.
- This enabled an incremental cutover rather than a big-bang one.
- Result phrasing: roughly 10x faster processing, service availability maintained, zero correctness drift on financial datasets.
- Supporting work: automated workflows with Python, Lambda, and Step Functions; Terraform-managed AWS infrastructure.

HP role record — highlights should include

- Scope: thousands to millions of customers, globally.
- Method: Domain-Driven Design AND event-driven architecture; Rails monolith to cloud-native microservices and data pipelines.
- Constraint: completed in a matter of months, in collaboration with 4 geographically distributed teams, with stability and service reliability maintained throughout.
- Note for the agent: DDD was both an architecture tool and a coordination tool for the distributed teams — this is a strong, senior-sounding point if asked a follow-up.

Canonical-phrasing note

Where the resume uses softer phrasing (“migrated targeted workloads”) and the Q&A material uses stronger, more specific phrasing (“led a migration… to solve a scaling and cost problem… 10x faster”), the knowledge pack should carry the Q&A version as the canonical highlight. The agent should reach for the stronger accurate phrasing by default.

9.2 Harden the sensitive questions (faq records)

Two questions are guaranteed to come up and carry real downside if the agent improvises: why Marwan left Apptio, and what he is looking for next. Both should be canned-basis entries in the faq records so the agent paraphrases a vetted answer rather than generating its own framing. Marwan's own wording is professional and non-bitter — that wording is the spec.

FAQ record — “Why did you leave Apptio?”

```
triggers: ["why did you leave Apptio", "why did he leave IBM",
           "what happened at Apptio", "why is he looking"]
```

FINAL COPY — CANNED-BASIS ANSWER (AGENT PARAPHRASES THIS, DOES NOT IMPROVISE)

Apptio was acquired by IBM during Marwan's time there. As the organization integrated, it shifted toward more process overhead and company-wide structural changes. He's looking for a role with more direct technical ownership.

FAQ record — “What are you looking for next?”

```
triggers: ["what is he looking for", "what does he want next",
           "what kind of role", "what's next for him"]
```

FINAL COPY — CANNED-BASIS ANSWER

Marwan is looking for a senior role where he can own systems end to end — architecture and platform work on data-intensive, cloud-native products. He works best with direct technical ownership and a high bar, and brings 15 years of doing exactly that at HP, JumpCloud, and Apptio.

Guardrail

The agent should never editorialize beyond these vetted answers on the departure question — no speculation about IBM, no negativity, no detail Marwan did not provide. If pushed for more, it should offer the contact form rather than improvise.

9.3 The guaranteed-question FAQ records

The hero's suggestion chips — “Platform architecture,” “Impressive accomplishment,” “Most challenging project” — guarantee three specific questions get asked. Each should have a tight

canonical FAQ answer rather than relying on retrieval, so the highest-traffic answers are consistent every time.

FAQ record — “Most impressive accomplishment?” (chip: Impressive accomplishment)

FINAL COPY — CANONICAL ANSWER

Marwan planned and executed the scaling of HP's Instant Ink subscription service from thousands of customers to millions, globally. He transformed a Ruby on Rails monolith into cloud-native microservices and data pipelines using Domain-Driven Design and Test-Driven Development — completed in a matter of months, across 4 geographically distributed teams, with service reliability maintained throughout.

FAQ record — “Most challenging technical problem?” (chip: Most challenging project)

FINAL COPY — CANONICAL ANSWER

At Apptio, Marwan migrated Kubernetes and Java workloads to EMR Spark with Scala to solve a scaling and cost problem — pipelines were running for days and getting expensive. The hard part was correctness: financial datasets have zero tolerance for drift. He built a reconciliation pipeline that ran both systems in parallel and flagged discrepancies, which gave the team confidence to cut over incrementally rather than all at once. The result was roughly 10x faster processing with zero correctness drift.

FAQ record — “Platform architecture experience?” (chip: Platform architecture)

FINAL COPY — CANONICAL ANSWER

Platform architecture is the core of Marwan's work. At HP he decomposed a Rails monolith into cloud-native microservices on AWS using Domain-Driven Design. At JumpCloud he architected a hybrid Postgres and MongoDB data layer for a multi-tenant identity platform serving 50,000+ tenants. At Apptio he designed and migrated data-processing pipelines for high-volume financial datasets. The through-line: clear service boundaries, operable systems, and designs that scale with both data and teams.

9.4 Recruiter skill-match retrieval

Marwan's Q&A material includes several “my experience maps to your job description” answers. Recruiters ask exactly these questions — “does he have GraphQL experience?”, “has he done high-volume data pipelines?” — and the agent should be able to map his background onto a stack confidently. This is mostly a tagging exercise: the right relevance tags on the right roles so tag-retrieval surfaces the right evidence.

Tagging — ensure these tags resolve to the right roles

Tag	Should retrieve	Key evidence the agent should cite
graphql	JumpCloud	Directed end-to-end design of a GraphQL M365 integration;

Tag	Should retrieve	Key evidence the agent should cite
		multi-tenant schema and resolvers; 50,000+ tenants.
aws	Apptio, HP	Lambda, Step Functions, EMR, Terraform (Apptio); Lambda, API Gateway, SNS, SQS, DynamoDB, Redshift (HP).
data-pipelines	Apptio, HP	High-volume financial data pipelines with zero drift (Apptio); event-driven data pipelines at subscription scale (HP).
multi-tenant	JumpCloud	Hybrid Postgres + MongoDB layer; 50,000+ tenants; per-tenant attribute isolation.
microservices	HP	Rails monolith to cloud-native microservices via Domain-Driven Design.
scala / spark	Apptio	Migrated K8s + Java workloads to EMR Spark with Scala; ~10x faster.
python	Apptio	Automated operational workflows with Python, Lambda, Step Functions.
mentoring / leadership	HP, AT&T	Led 4 distributed teams (HP); mentored new developers, raised code-quality bar (AT&T).

bio record — add the AI-tooling fact

“Does he use AI tools?” is increasingly a real interview question. The bio record should let the agent answer it specifically and confidently.

FINAL COPY — BIO RECORD, AI-TOOLING ENTRY

Marwan works with AI-assisted tooling daily: planning large initiatives and generating documentation with Claude, prototyping and writing code in Cursor, and using multi-model workflows to move faster without lowering the quality bar. This portfolio's chat agent is itself an example of that practice.

9.5 Agent behavior — confirmed good, and one watch item

From the live agent behavior observed during review:

Confirmed correct — keep as-is

- Third-person voice. “At Apptio, Marwan led a migration…” — correct. The agent speaks about Marwan, never as him.

- Accuracy. The EMR Spark answer reproduces the reconciliation-pipeline detail faithfully and does not invent metrics.
- Scope discipline. The panel subtitle (“Grounded in my resume and projects. Not a general chatbot.”) sets the right expectation and the agent honors it.

Watch item — canonical phrasing drift

Verify the agent reaches for the Q&A material's stronger framing (“led a migration to solve a scaling and cost problem… 10x faster”) rather than the resume's softer phrasing (“migrated targeted workloads… to improve performance”). Both are accurate; the stronger one is the better answer and should be the canonical highlight in the knowledge pack. See 9.1.

Watch item — model string exposure

The chat panel currently displays “Model: gemini-3.1-flash-lite” to the visitor. This is debug-grade detail that slightly undercuts the “grounded in my resume” framing and reads as something left switched on. Remove it from the user-facing panel, or, if transparency is wanted, move it behind an unobtrusive info affordance rather than printing it inline. Tracked in the punch list, Section 10.

10. Open punch list — issues copy cannot fix

These are the visual and technical issues surfaced across the recent reviews that completed copy does not address. They are ordered by impact. Item 1 is the most damaging: it makes a working feature look broken, and it has persisted across multiple review rounds.

Priority	Issue	Fix
1 — High	Chat panel is semi-transparent. The hero and portfolio cards bleed through the panel; in places the spaceman renders on top of message text. Makes a working feature look broken.	Give the panel an opaque (or near-opaque + heavy backdrop-blur) background; darken the backdrop layer enough to fully mute everything behind it.
2 — High	Panel floats low and off-center, with dead space above it and near-contact with the footer. Position drifts with message count.	Vertically center the panel in the viewport; set a fixed max-height with internal scroll.
3 — High	“Model: gemini-3.1-flash-lite” printed inline in the panel. Debug-grade detail, reads as left-on.	Remove from the user-facing panel, or move behind an unobtrusive info affordance. See 9.5.
4 — Med	Slim header chatbar on sub-pages collides with / evicts the centered logo.	Move the logo left next to “Home,” or stop the chatbar occupying dead center.
5 — Med	Accent color is split — blue for send button and user bubbles, purple for focus rings and the “ASK ANYTHING” dot.	Commit to one accent across all actionable elements.
6 — Med	Hero is too tall. Proof line + divider + company strip push the Resume/Contact CTAs far down the page.	Tighten proof-line copy (drop “from thousands to”); reduce vertical spacing between proof line, company strip, and chat input.
7 — Med	Mobile hero ordering buries the chat input and pushes CTAs off-screen.	On mobile, move Resume/Contact directly under the tagline, or collapse the proof line to one line.
8 — Med	Spaceman thought bubble clips on narrow widths and overlaps the open panel.	Add a max-width to the bubble; reposition so it cannot collide with the panel.
9 — Med	Deployed static index.html is stale — server HTML still shows the old hero and old positioning; new content is JS-injected. Crawlers and slow loads see the outdated version.	Sync the deployed static markup to the current hero; update the meta description to include “Senior… 15+ years.”
10 — Low	“Playground” label undercuts the senior-architecture positioning.	Rename to “Experiments,” or order it after Portfolio in the nav. Content stays.

If only three things get done

Items 1, 2, and 3. Panel transparency and positioning are polish that reads as a bug; the model string is a thirty-second fix. Clearing those three lifts the whole site a tier.

11. Implementation checklist

A consolidated, ordered task list across copy, content, and fixes.

Copy & content (drop-in from this document)

1. Add the About / experience summary block between hero and Portfolio (Section 4).
2. Fill the three portfolio detail modals with the case studies (Section 5).
3. Restructure the two Playground cards; fill the two Playground detail modals (Section 6).
4. Add the earlier-career line below the portfolio cards (Section 7.1).
5. Add the contact form subtitle and confirmation-state message (Section 7.2).
6. Add the chat panel empty-state line (Section 7.3).
7. Add the streaming-state spaceman phrase and the extra ambient phrases (Section 7.4).
8. Confirm the AT&T portfolio card is deployed (Section 3).

Hero & agent consolidation (Section 8)

1. Merge the spaceman thought-bubble and the hero chat bar into one global agentNode element with a data-state machine (Section 8.3).
2. Implement the bubble-to-bar CSS morph driven off [data-state] (Section 8.3).
3. Rewrite the modal open/close to expand from the agentNode's bounding rect, preserving all current modal mechanics (Section 8.3).
4. Set page-dependent resting state: bubble on the home hero, bar on sub-pages (Section 8.2).
5. Re-anchor the dot-trail to the agentNode; keep it mounted across all three states (Section 8.3).
6. Confirm the agentNode subscribes to chatBus so agent status overrides the morph state (Section 8.1).

Agent knowledge base (validate against repo data files)

1. Sharpen Apptio and HP role records with the Q&A-material specifics (Section 9.1).
2. Add the two sensitive-question FAQ records: leaving Apptio, looking-for-next (Section 9.2).
3. Add the three guaranteed-question FAQ records tied to the hero chips (Section 9.3).
4. Verify relevance tags resolve to the right roles for recruiter skill-match (Section 9.4).
5. Add the AI-tooling fact to the bio record (Section 9.4).
6. Verify canonical-phrasing default favors the stronger accurate framing (Section 9.5).

Fixes (Section 10, in priority order)

1. Make the chat panel opaque; darken the backdrop.
2. Center the panel; set a fixed max-height with internal scroll.
3. Remove the inline model string from the panel.
4. Resolve the slim chatbar / logo collision on sub-pages.
5. Unify the accent color across all actionable elements.
6. Tighten the hero height; fix mobile ordering.
7. Fix the spaceman thought-bubble clipping and overlap.

8. Sync the deployed static markup and meta description.
9. Reconsider the “Playground” label.

End of document. Sections 3 through 7 are drop-in copy. Section 8 is the hero & agent consolidation design. Section 9 is a specification to validate against the knowledge files in github.com/geda0/gvp. Section 10 is the open punch list. Everything traces to the resume, the Q&A material, or observed live behavior.